

# SYSTEM DESIGN REPORT

## AI-BASED FAKE PROFILE DETECTION

### 1. ARCHITECTURE OVERVIEW

The system follows a layered architecture: presentation, API, processing, machine-learning, persistence and monitoring.

### 2. MODULES

Data Collection Module: receives profile attributes.

Preprocessing Module: cleans and transforms data.

Feature Engineering Module: calculates derived behavioral indicators.

Model Module: loads the trained classifier.

Risk Module: converts probability to an operational risk category.

Explainability Module: identifies influential features.

Database Module: stores predictions and reviews.

Dashboard Module: displays results.

### 3. DATA FLOW

Input → Validation → Cleaning → Feature Engineering → Model Prediction → Risk Scoring → Explanation → Storage/Display.

### 4. UML DESIGN

Use Case actors include Administrator and Analyst.

Core use cases include Submit Profile, Predict Risk, View Explanation, Review Alert and Export Report.

Activity flow begins with authentication, continues through profile submission and analysis, and ends with review or export.

Sequence flow connects Dashboard, API, Feature Engine, Model and Explanation Service.

### 5. DATABASE DESIGN

Profile(profile\_id, account\_age, followers, following, posts, verification).

Prediction(prediction\_id, profile\_id, model\_version, probability, risk, timestamp).

Explanation(explanation\_id, prediction\_id, feature, contribution).

Review(review\_id, prediction\_id, reviewer, decision, comments).

## **6. API DESIGN**

POST /predict – receives profile features and returns prediction.

GET /prediction/{id} – retrieves a stored prediction.

POST /review – stores an analyst review.

GET /health – service health check.

## **7. DEPLOYMENT**

Docker containers can separate frontend, API, database and model service. Environment variables should store configuration and secrets.

## **8. SECURITY**

Use authentication, authorization, TLS, input validation, rate limiting, secure logging and least-privilege database access.

## **9. SCALABILITY**

Stateless API instances can be scaled horizontally. Model artifacts can be versioned. Heavy batch analysis can be moved to asynchronous workers.

## **10. CONCLUSION**

The modular design allows independent development and future replacement of algorithms without redesigning the entire system.